УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 18

СОЗДАНИЕ ПРОСТЕЙШЕГО UDP-СЕРВЕРА И ПРОСТЕЙШЕГО UDP-КЛИЕНТА, КОНСОЛЬНОГО КЛИЕНТ-СЕРВЕРНОГО ПРИЛОЖЕНИЯ НА ОСНОВЕ ДЕЙТАГРАММНЫХ СОКЕТОВ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейшего UDP-сервера и простейшего UDP-клиента, консольного клиент-серверного приложения на основе дейтаграммных сокетов

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Работа с UDP сокетами в Python

UDP (User Datagram Protocol) — транспортный протокол, обеспечивающий передачу данных без установления соединения.

В отличие от TCP, UDP является протоколом без установления соединения (connectionless). Это означает, что перед отправкой данных не требуется устанавливать специальное соединение между клиентом и сервером. Данные просто отправляются в виде независимых пакетов (датаграмм) по сети.

Преимущества UDP:

– быстрота: Отсутствие установления соединения и подтверждения доставки делает UDP быстрее, чем TCP;

– меньшая нагрузка: Меньше служебной информации (overhead) в заголовках пакетов.

Недостатки UDP:

– ненадежность: Данные могут быть потеряны, дублированы или доставлены в неправильном порядке. UDP не предоставляет механизмов для восстановления потерянных пакетов или упорядочивания их;

– отсутствие гарантии доставки: Нельзя быть уверенным, что пакет достигнет получателя.

4.2 Создание UDP сокетов в Python

Для работы с UDP в Python, как и с TCP, используется модуль socket.
socket.socket(socket.AF_INET, socket.SOCK_DGRAM):

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Эта строка создает UDP-сокет.

socket.AF_INET: Указывает на использование IPv4 адресов.

socket.SOCK_DGRAM: Указывает на использование UDP протокола. DGRAM расшифровывается как Datagram, что отражает суть UDP.

4.3 Серверная часть (UDP)

socket.bind((host, port)):

HOST = 'localhost'

PORT = 12345

s.bind((HOST, PORT))

Привязывает сокет к определенному адресу и порту. Сервер будет прослушивать входящие UDP-пакеты на этом адресе и порту.

socket.recvfrom(buffersize):

```
BUFFER_SIZE = 1024
data, address = s.recvfrom(BUFFER_SIZE)
print(f"Получено сообщение '{data.decode('utf-8')}' от {address}")
```

Получает данные и адрес клиента, отправившего данные.

data: Полученные данные (в виде байтов).

address: Адрес клиента (IP-адрес и номер порта).

buffersize: Максимальный размер буфера для приема данных. Важно выбрать размер, достаточный для приема ожидаемых UDP-пакетов.

socket.sendto(data, address):

```
message = "Эхо: " + data.decode('utf-8')
s.sendto(message.encode('utf-8'), address)
```

Отправляет данные клиенту по указанному адресу. Адрес клиента был получен при вызове recvfrom().

socket.close():

s.close()

Закрывает сокет.

UDP предоставляет более быстрый способ передачи данных без гарантий доставки. Это делает его подходящим для приложений реального времени.

UDP-сервер

```
import socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) #Создание UDP сокета
server_socket.bind(('localhost', 54321)) # Привязка к адресу
print("UDP-сервер запущен...")
while True:
    data, addr = server_socket.recvfrom(1024) # Получение данных и адреса отправителя
    print(f"Получено сообщение от {addr}: {data.decode('utf-8')}")
    server_socket.sendto("Ответ от сервера".encode('utf-8'), addr) # Отправка ответа
```

4.4 Клиентская часть (UDP)

socket.sendto(data, (host, port)):
HOST = 'localhost'
PORT = 12345
MESSAGE = "Привет, UDP сервер!"
s.sendto(MESSAGE.encode('utf-8'), (HOST, PORT))
Отправляет данные серверу. Указывается адрес и порт сервера. Здесь не нужно предварительно устанавливать соединение.
socket.recvfrom(buffersize) (опционально):
BUFFER_SIZE = 1024
data, address = s.recvfrom(BUFFER_SIZE)
print(f"Получено сообщение '{data.decode('utf-8')}' от {address}")
Этот вызов необязателен. Он позволяет клиенту получить ответ от сервера. Например, в нашем эхо-сервере клиент получает повторенное сообщение от сервера. Обратите внимание, что клиент не обязан получать ответ. UDP позволяет отправлять данные в одном направлении.
socket.close():
s.close()
Закрывает сокет.
UDP-клиент

```
import socket
# Создание UDP клиента
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# Отправка данных
client_socket.sendto("Привет, UDP-сервер!".encode('utf-8'), ('localhost',
54321))
# Получение ответа
data, addr = client_socket.recvfrom(1024)
print(f"Ответ сервера: {data.decode('utf-8')}")
client_socket.close()
```

UDP использует методы sendto() и recvfrom() вместо send() и recv(). Это связано с тем, что UDP не устанавливает постоянное соединение между клиентом и сервером.

4.5 Примеры кода UDP Socket на Python

Простой UDP-сервер, повторяющий полученные данные:
server.py:

```
import socket
HOST = 'localhost'
PORT = 12345
BUFFER_SIZE = 1024
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind((HOST, PORT))
```

```
print(f"UDP сервер запущен на {HOST}:{PORT}")
while True:
    data, address = s.recvfrom(BUFFER_SIZE)
    message = "Эхо: " + data.decode('utf-8')
    s.sendto(message.encode('utf-8'), address)
    print(f"Отправлено эхо-сообщение '{message}' клиенту {address}")
```

client.py:

```
import socket
HOST = 'localhost'
PORT = 12345
MESSAGE = "Привет, UDP сервер!"
BUFFER_SIZE = 1024
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(MESSAGE.encode('utf-8'), (HOST, PORT))
data, address = s.recvfrom(BUFFER_SIZE) # Ждем ответа от сервера
print(f"Получено сообщение '{data.decode('utf-8')}' от {address}")
s.close()
```

В коде выше мы реализовали простой UDP-клиент, отправляющий сообщение на сервер. Если вам нужен клиент, который просто отправляет сообщение и не ждет ответа, уберите строки с recvfrom.

4.6 Пример решения задачи на языке программирования Python

Задача. Напишите консольное клиент-серверное приложение на основе UDP-сокетов, в котором клиент отправляет серверу строку текста. Сервер принимает сообщение, подсчитывает количество слов в полученной строке и отправляет результат обратно клиенту. Клиент должен вывести полученное количество слов на экран.

Решение задачи:

Сервер (server.py)

```
Сервер (udp_server.py)
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(('localhost', 5000))
print("UDP-сервер запущен")
while True:
    data, address = server.recvfrom(1024)
    message = data.decode()
    print("Получено:", message)
    count = len(message.split())
    server.sendto(str(count).encode(), address)
```

Клиент (udp_client.py)

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = input("Введите строку: ")
```

```
client.sendto(message.encode(), ('localhost', 5000))
data, address = client.recvfrom(1024)
print("Количество слов:", data.decode())
client.close()
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите UDP-сервер, принимающий одно текстовое сообщение от клиента и выводящий его в консоль.

2 Напишите UDP-клиент, отправляющий серверу текстовую строку, введённую пользователем с клавиатуры.

3 Напишите UDP-сервер, который после получения сообщения отправляет клиенту ответ «Сообщение получено».

4 Напишите UDP-клиент, принимающий ответ сервера и выводящий его в консоль.

5 Напишите UDP-сервер, принимающий несколько сообщений от разных клиентов и выводящий их по порядку получения.

6 Напишите UDP-клиент, отправляющий серверу три сообщения с небольшими паузами между отправками.

7 Напишите UDP-сервер, который подсчитывает количество полученных сообщений и отправляет клиенту номер принятого сообщения.

8 Напишите UDP-клиент, который получает от сервера номер принятого сообщения и отображает его пользователю.

9 Напишите UDP-сервер, преобразующий полученное сообщение в верхний регистр и отправляющий результат клиенту.

10 Напишите UDP-клиент, который отправляет серверу строку и выводит полученную преобразованную строку.

11 Напишите UDP-сервер, принимающий сообщение и возвращающий его длину в символах.

12 Напишите UDP-клиент, отправляющий строку и выводящий длину сообщения, полученную от сервера.

13 Напишите UDP-сервер, который завершает работу после получения специального сообщения «exit».

14 Напишите UDP-сервер, который принимает от клиента несколько чисел, вычисляет их сумму и отправляет результат обратно клиенту.

15 Напишите UDP-клиент, который отправляет серверу строку и получает количество слов в этой строке, вычисленное сервером.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы
6.2 Цель лабораторной работы
6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите, что такое протокол UDP в языке программирования Python.
7.2 Сравните, основные отличия протоколов UDP и TCP в языке программирования Python.
7.3 Охарактеризуйте, что такое дейтаграммный сокет в языке программирования Python.
7.4 Перечислите, какие функции используются для отправки и приёма данных в UDP в языке программирования Python.
7.5 Объясните, почему UDP работает быстрее TCP в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.
2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.
3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.